# Laboratoare AA
## Reduceri Turing

### Probleme decidabile

→ probleme de decizie: $f: \Sigma^* \to \{0,1\}$

→ MT ≃ algoritm

M acceptă $w$ $M(w) = TRUE$ dacă M pe inputul $w$ se termină și pe banda se află "TRUE"

$M(w) \to TRUE$

M acceptă $f \iff (M(w) \to TRUE \iff f(w) = TRUE)$

$f(w) = FALSE \iff M(w) \to 0$ sau $M(w)$ ciclează

M decide $f \iff \begin{pmatrix} M(w) \to TRUE \iff f(w) = TRUE \\ M(w) \to FALSE \iff f(w) = FALSE \end{pmatrix}$

M decide $f \Rightarrow$ M acceptă $f$

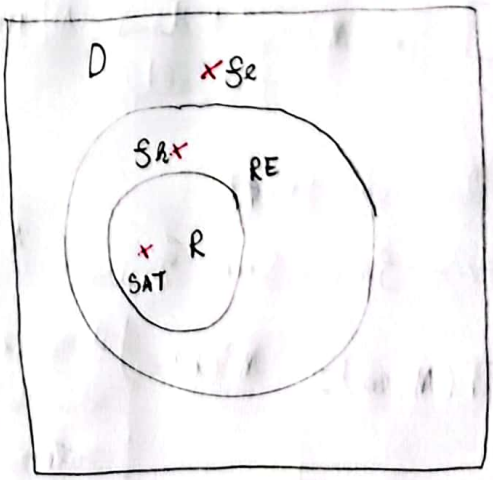
$R = \{f \mid f: \Sigma^* \to \{0,1\} \mid \exists$ o M.T. M care decide $f\}$

↑ mulțimea problemelor decidabile

$RE = \{f: \Sigma^* \to \{0,1\} \mid \exists$ o M.T. M care acceptă $f\}$

↑ mulțimea pb. acceptabile

$R \subset RE$




$f_R(M,w) = \begin{cases} 1, \text{dacă } M(w) \text{ se termină} \\ 0, \text{altfel} \end{cases}$

Halting Problem

$f_\ell(M,w) = \begin{cases} 1, \text{dacă } M(w) \text{ ciclează} \\ 0, \text{altfel} \end{cases}$

Looping Problem

$\overline{f_R} = f_\ell$

Scanat cu CamScanner

# Reduceri Turing

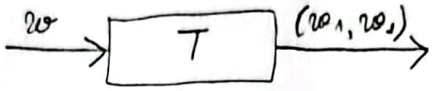
$f \leq_m g$ dacă $\forall w \in \Sigma^*$, $f(w) = g(t(w))$ și $t: \Sigma^* \to \Sigma^*$ poate fi implementată de o NT care se termină mereu

Ex. $f(w) = \begin{cases} 1, \text{ dacă } w \text{ e palindrom} \\ 0, \text{ altfel} \end{cases}$

$g(w_1, w_2) = \begin{cases} 1, \text{ dacă } w_1 == \text{reverse}(w_2) \\ 0, \text{ altfel} \end{cases}$

23:21

Dem. că $f \leq_m g$




Fiem $M_g$ care decide $g$. Construim $M_f$ care decide $f$.

$M_f(w)$ { scrie pe bandă $w, w$
rulează $M_g$

$T(w) = (w, w)$

```
def g(w1, w2):
    ----> return w1 == reverse(w2)
def h(w):
    return g(t(w))
def T(w):
    w1 = w
    w2 = w
    return (w1, w2)
```

**Teoreme:**
$f \leq_m g$ și $g \in R \Rightarrow f \in R$
$f \leq_m g$ și $f \notin R \Rightarrow g \notin R$

Ex. Dacă trebuie să $g \notin R$: găsim $f$ a.î. $f \leq_m g$ și $f \notin R \Rightarrow g \notin R$

1. $f_{111}(M) = \begin{cases} 1, \text{dacă } M[111] \to \text{TRUE} \\ 0, \text{altfel} \end{cases}$

Dem. că $f_{111} \notin R$

$M'(111) \to \text{TRUE}$ ⇕ $t(M, w)$

—//— $f_A \leq_m f_{111}$. Găsim 

și arătăm că $f_A(M, w) = 1 \iff f_{111}(t(M, w)) = 1$

$T(M, w) =$ { $M'(x) =$ { șterge $x$ de pe bandă
scrie $w$ pe bandă
rulează $M$
dă răspunsul "TRUE" } $\forall$ input pe primul, $M'$ rulează $M[w]$ și întoarce TRUE

„$\Rightarrow$" $f_A(M, w) = 1 \Rightarrow M(w)$ se termină $\to \forall x$, $M'(x) \to \text{TRUE} \Rightarrow M'[111] \to \text{TRUE} \Rightarrow$

$\Rightarrow f_{111}(M') = 1$ $\quad M' = t(M, w)$

Scanat cu CamScanner

„⇐" $f_{111}(M')=1 \Rightarrow M'[111] \to$ TRUE $\Rightarrow M[w]$ se termină $\Rightarrow f_R(M,w)=1 \Rightarrow$

Am arătat că $f_R \leq_m f_{111}$ / $\Rightarrow f_{111} \notin R$
dar $f_R \notin R$ /

② $f_P(M) = \begin{cases} 1, \text{dacă } \forall w, M[w] \text{ decide dacă } w \text{ e palindrom} \\ 0, \text{altfel} \end{cases}$

—//— $f_P \notin R \Rightarrow$ —//— $f_R \leq_m f_P$




$T(M,w) = \begin{cases} M'(x) = \begin{cases} \text{șterge } x \\ \text{scrie } w \\ \text{rulează } M \\ \text{scrie } x \\ \text{verifică dacă } x \text{ e palindrom} \end{cases} \end{cases}$

—//— $f_R(M,w)=1 \Leftrightarrow f_P(M')=1$

„⇒" $f_R(M,w)=1 \Rightarrow M[w]$ se termină $\Rightarrow M'[x]$ decide dacă $x$ e palindrom $\Rightarrow$
$\Rightarrow f_P(M')=1$

„⇐" $f_P(M')=1 \Rightarrow M'[x]$ decide dacă $x$ e palindrom $\Rightarrow M[w]$ se termină $\Rightarrow$
$\Rightarrow f_R(M,w)=1$

③ $f_{rev}(M) = \begin{cases} 1, \text{dacă } \forall w, M[w] = reverse(w) \\ 0, \text{altfel} \end{cases}$ ← mașina îmi dă reverse(w)

—//— $f_{rev} \notin R \Rightarrow$ —//— $f_R \leq_m f_{rev}$

$T(M,w) = \begin{cases} M'[x] = \begin{cases} \text{șterge } x \\ \text{scrie } w \\ \text{rulează } M \\ \text{return } reverse(x) \end{cases} \end{cases}$ } rulează $M[w]$

outputul nu e deloc condiționat de M, doar ajutorul lui M' da

—//— $f_R(M,w)=1 \Leftrightarrow f_{rev}(M')=1$

„⇒" $f_R(M,w)=1 \Rightarrow M[w]$ se termină $\Rightarrow M'[x]=reverse(x), \forall x \Rightarrow f_{rev}(M)=1$

„⇐" $f_{rev}(M')=1 \Rightarrow M'[x]$ se termină $\forall x \Rightarrow M[w]$ se termină $\Rightarrow f_R(M,w)=1$ / $\Rightarrow$

$\Rightarrow f_R \leq_m f_{rev}$ / $\Rightarrow f_{rev} \notin R$

3) dar $f_R \notin R$

Scanat cu CamScanner

4 $f_{oracol}(M) = \begin{cases} 1, \text{dacă } M[enc(M)] = 1 \\ 0, \text{altfel} \end{cases}$

—//— $f_{oracol} \notin R$ (Să știi poezia clasică de început)

$f_R \leq_m f_{oracol}$

$t(M, w) = \left| M'[x] = \begin{cases} \text{rulează } M[w] \\ \text{if } x == enc(M) \\ \quad \text{return 1} \\ \text{else} \\ \quad \text{ciclează} \end{cases} \right.$ pot înlocui și return 1, tot e corect

—//— $f_R(M, w) = 1 \iff f_{oracol}(M') = 1$

$M[w]$ se termină $\iff M'[enc(M')] \to TRUE$

„$\Rightarrow$" $f_R(M, w) = 1 \rightsquigarrow M[w]$ se termină $\Rightarrow M'[enc(M')] \to TRUE \Rightarrow f_{oracol}(M') = 1$

„$\Leftarrow$" $f_{oracol}(M') = 1 \Rightarrow M'[enc(M')] \to TRUE \Rightarrow M[w]$ se termină $\Rightarrow f_R(M, w) = 1$ / $\Rightarrow$

$\Rightarrow f_R(M, w) = 1 \iff f_{oracol}(M') = 1 \Rightarrow f_R \leq_m f_{oracol}$ dar $f_R \notin R$ / $\Rightarrow f_{oracol} \notin R$

5 $f_{finite}(M) = \begin{cases} 1, \text{dacă } M \text{ se oprește pt. un nr. finit de cuvinte} \\ 0, \text{altfel} \end{cases}$

Ce urmărim: I oprirea lui M' să fie condiționată de oprirea lui M
II pt. un nr finit de cuvinte să se oprească

—//— $f_{finite} \notin R$.

Pentru a arăta că $f_{finite} \notin R$, vom încerca să demonstrăm că $f_R \leq_m f_{finite}$. Găsim o transformare $t: \Sigma^* \to \Sigma^*$ care poate fi implementată de o MT care se termină mereu astfel încât $f(w) = g(t(w)), \forall w \in \Sigma^*$ și arătăm că $f_R(M, w) = 1 \iff f_{finite}(M') = 1$

$t(M, w) = \left| M[x] = \begin{cases} \text{rulează } M[w] \\ \text{dacă } x == w \\ \quad \text{return 1} \\ \text{else} \\ \quad \text{ciclează} \end{cases} \right.$

„$\Rightarrow$" $f_R(M, w) = 1 \Rightarrow M[w]$ se oprește $\Rightarrow M'$ se oprește pt. exact 1 input $\Rightarrow f_{finite}(M') = 1$ /

„$\Leftarrow$" $f_{finite}(M') = 1 \Rightarrow M'$ se oprește pentru un număr finit de cuvinte $\Rightarrow M[w]$ se oprește $\Rightarrow f_R(M, w) = 1$ /$\Rightarrow$

$\Rightarrow f_R \leq_m f_{finite}$ dar $f_R \notin R$ / $\Rightarrow f_{finite} \notin R$



Scanat cu CamScanner

$f_{set}(A, M) = \begin{cases} 1, \text{ dacă } M[w] \text{ se oprește } \forall w \in A \\ 0, \text{ altfel} \end{cases}$

↓ limbaj finită

$f_{set} \notin R$

$-//- \; f_R \leq_m f_{set}$

$T(M, w) = \begin{cases} A = \{w\} \\ M'[x] = \begin{cases} \text{rulează } M[x] \\ \text{return 1} \end{cases} \end{cases}$

$-//- \; f_R(M, w) = 1 \iff f_{set}(A, M') = 1$

$\Updownarrow$

$M[w]$ se termină $\iff$ $M'$ se termină pt. $\forall$ cuvânt din $A$

„$\Rightarrow$" $f_R(M, w) = 1 \Rightarrow M[w]$ se termină $\Rightarrow M'$ se termină pt. $\forall$ cuvânt din $A = \{w\} \Rightarrow$

$\Rightarrow f_{set}(A, M') = 1$

„$\Leftarrow$" $f_{set}(A, M') = 1 \Rightarrow M'$ se termină pt. $\forall$ cuvânt din $A \rightarrow M[w]$ se termină $\Rightarrow f_R(M, w) =$

$\Rightarrow f_R(M, w) = 1 \iff f_{set}(A, M') = 1 \Rightarrow f_R \leq_m f_{set}$ / $\Rightarrow f_{set} \notin R$

dar $f_R \notin R$

---

7 $f_x(M) = \begin{cases} 1, \text{ dacă } \exists w \text{ pt. care } M[w] \text{ scrie } x \text{ pe bandă} \\ 0, \text{ altfel} \end{cases}$

$f_R \leq_m f_x$

$T(M, w) = \begin{cases} M'[x] = \begin{cases} \text{rulează } M[w] \\ \text{dacă } x == w \\ \quad \text{scrie 'x'} \\ \quad \text{return 1} \\ \text{else} \\ \quad \text{loop} \end{cases} \\ \text{scrie 'x'} \\ \text{return 1} \end{cases}$

$-//- \; f_R(M, w) = 1 \iff f_x(M') = 1$

$\Updownarrow \qquad \Updownarrow$

$M[w]$ se termină $\iff \exists w$ pt. care $M[w]$ scrie $x$ pe bandă

„$\Rightarrow$" $f_R(M, w) = 1 \Rightarrow M[w]$ se termină $\Rightarrow M'[w]$ scrie $x$ pe bandă $\Rightarrow \exists$ un input $y$ pt. care

$M'[y]$ scrie '$x$' $\Rightarrow f_x(M') = 1$

„$\Leftarrow$" $f_x(M') = 1 \Rightarrow M'$ scrie $x$ pe bandă pt. 1 input $\rightarrow M[w]$ se termină $\Rightarrow f_R(M, w) = 1$



Scanat cu CamScanner

8 $f_{eq}(M_1, M_2) = \begin{cases} 1, \text{ dacă } \forall w,\ M_1(w) = M_2(w) \\ 0, \text{altfel} \end{cases}$

$\nrightarrow f_{eq} \notin R$

$f_R \leq_m f_{eq}$

$T(M, w) = \begin{cases} M_1[x] = \begin{cases} \text{rulează } M[w] \\ \text{return 1} \end{cases} \\ M_2[x] = \{ \text{return 1} \end{cases}$

$f_R(M, w) = 1 \iff f_{eq}(M_1, M_2) = 1$

$\Uparrow \qquad \Uparrow$

$M[w]$ se termină $\iff M_1[w] \equiv M_2[w],\ \forall w \in \Sigma^*$

„$\Rightarrow$" $f_R(M, w) = 1 \Rightarrow M[w]$ se termină $\Rightarrow M_1[x] = 1,\ \forall x$

dar, din construcție $M_2[x] = 1,\ \forall x$ $\Big| \Rightarrow M_1[x] \equiv M_2[x],\ \forall x \Rightarrow$

$\Rightarrow f_{eq}(M_1, M_2) = 1$

„$\Leftarrow$" $f_{eq}(M_1, M_2) = 1 \Rightarrow M_1[x] \equiv M_2[x],\ \forall x$

dar $M_2[x] = 1,\ \forall x$ $\Big| \Rightarrow M_1[x] = 1,\ \forall x \Rightarrow M[w]$ se termină $\Rightarrow f_R(M, w) = 1$

$\Rightarrow f_R \leq_m f_{eq}$

dar $f_R \notin R$ $\Big| \Rightarrow f_{eq} \notin R$

ⓐ

Scanat cu CamScanner

# Laboratoare AA

## R, RE, Reduceri Turing

$f \in R \Leftrightarrow \exists$ o M.T. M care decide $f$ $\Leftrightarrow$ M se termină pt. $\forall$ input și $f(w)=1 \Leftrightarrow M[w]=1$ și $f(w)=0 \Leftrightarrow M[w]=0$

$f \in RE \Leftrightarrow \exists$ o M.T. care acceptă $f$ $\Leftrightarrow f(w)=1 \Leftrightarrow M[w]=1$ și $f(w)=0 \Leftrightarrow M[w]=0$ sau $M[w]$ ciclează

$f \leq_m g \Leftrightarrow \exists$ o transformare $t: \Sigma_1^* \to \Sigma_1^*$ a.î. $f(w) = g(t(w))$

---

**1** $f_{all} \leq_m f_{eq}$

$$f_{all}(M) = \begin{cases} 1, \text{ dacă } \forall w, M[w] \text{ se oprește} \\ 0, \text{ altfel} \end{cases}$$

$$f_{eq}(M_1, M_2) = \begin{cases} 1, \text{ dacă } \forall w, M_1[w] \equiv M_2[w] \\ 0, \text{ altfel} \end{cases}$$

de ce nu $w$?

$$t(M) = \begin{cases} M_1[x] = \{ \text{run } M[x] \text{ return 1} \\ M_2[x] = \{ \text{return 1} \end{cases}$$

$\Leftarrow$ " $f_{all}(M)=1 \Leftarrow f_{eq}(M_1, M_2)=1$

" $\Rightarrow$ " $f_{all}(M)=1 \Rightarrow M[x]$ se oprește $\forall x \to M_1[x]=1, \forall x$

dar din construcție $M_2[x]=1, \forall x$ $\Big| \Rightarrow M_1[x] \equiv M_2[x], \forall x$

$\Rightarrow f_{eq}(M_1, M_2)=1$ (*)

" $\Leftarrow$ " $f_{eq}(M_1, M_2)=1 \Rightarrow M_1[x] \equiv M_2[x], \forall x$

dar, din construcție, $M_2[x]=1, \forall x$ $\Big| \Rightarrow M_1[x]=1, \forall x \Rightarrow M[x]$ se termină $\forall x$

$\Rightarrow f_{all}(M)=1$ (**)

Din (*) și (**) $\Rightarrow f_{all} \leq_m f_{eq}$

---

! **2** $f_{any} \leq_m f_{eq}$

$$f_{any}(M) = \begin{cases} 1, \text{dacă } \exists w \text{ a.î. } M[w] \text{ se oprește} \\ 0, \text{altfel} \end{cases}$$

$$t(M) = \begin{cases} M_1[x] = \{ \text{șterge } x \text{ de pe bandă} \\ \quad i=1 \\ \quad \text{while(1)} \{ \\ \quad\quad \text{for } w \text{ in } (\Sigma^*) \text{ if } M[w] \text{ se oprește în } i \text{ pași} \\ \quad\quad\quad \text{return 1} \\ \quad\quad i{+}{+} \\ \quad \} \\ M_2[x] = \{ \text{return 1} \end{cases}$$

→ mulțime infinită

→ nu se ajunge aici

AȘA NU

Scanat cu CamScanner

transform în $f_{eq}$ pentru a mărea o mașină a răsp. clar

—//— $f_{any}(M) = 1 \iff f_{eq}(M_1, M_2) = 1$

—//— $\exists w$ pt. care $M[w]$ se oprește $\iff \forall x, M_1[x] \equiv M_2[x]$

$t(M) =$
- $M_1[x] =$
  - șterge x
  - i = 0
  - while true
    - for w in $\sum^0 + \sum^1 + \sum^2 + \dots + \sum^i$ (mult. cuv. de lungime maxim i)
      - if $M[w]$ se oprește în i pași
        - return 1
  - i++
- $M_2[x] =$
  - return 1

Să zicem că M ciclează pt. $\forall x$ mai puțin pt. $w$, $lung(w) = k$, $M[w]$ se oprește după $l$ pași

Când $i = max(k, l)$, îl găsesc pe $w$

„$\Rightarrow$” $f_{any}(M) = 1 \Rightarrow \exists w$ pentru care $M[w]$ se oprește $\Rightarrow M_1[x] = 1, \forall x$

dar, din construcție, $M_2[x] = 1, \forall x$ $\Big| \Rightarrow$

$\Rightarrow f_{eq}(M_1, M_2) = 1$

„$\Leftarrow$” $f_{eq}(M_1, M_2) = 1 \Rightarrow M_1[x] = 1, \forall x \Rightarrow \exists w$ a.î. $M[w]$ se oprește $\Rightarrow f_{any}(M) = 1$

**3** $f_P \leq_m f_{eq}$

$f_P(M) =$
- 1, dacă $\forall w$, $M[w]$ decide dacă $w$ e palindrom
- 0, altfel

$t(M) =$
- $M_1[x]$:
  - rulează $M[x]$
- $M_2[x]$:
  - dacă $x == rev(x)$
    - return 1
  - else
    - return 0

—//— $f_P(M) = 1 \iff f_{eq}(M_1, M_2) = 1$

„$\Rightarrow$” $f_P(M) = 1 \Rightarrow$ M decide dacă inputul e palindrom

dar, din construcție, $M_2$ decide dacă nr. e palindrom $\Big| \Rightarrow M_1[x] \equiv M_2[x], \forall x$

$\Rightarrow f_{eq}(M_1, M_2) = 1$

„$\Leftarrow$” $f_{eq}(M_1, M_2) = 1 \Rightarrow$ M decide dacă inputul e palindrom $\rightarrow f_P(M) = 1$

Scanat cu CamScanner

$\text{prog} \leq_m f_{eq}$

$$t(M) = \begin{cases} M_1[x] = \{ \text{prog } M[enc(M)] \\ M_2[x] = \{ \text{return } 1 \end{cases}$$

$$f_{prog}(M) = \begin{cases} 1, \text{ dacă } M[enc(M)] = 1 \\ 0, \text{ altfel} \end{cases}$$

prog M[w]
if w == enc(M)
return 1

↳ logic: se oprește

oare e echivalent cu ce e în stânga?

$f_{prog}(M) = 1 \iff f_{eq}(M_1, M_2) = 1$

„$\Rightarrow$" $f_{prog}(M) = 1 \Rightarrow M[enc(M)] = 1 \Rightarrow M_1[x] = 1, \forall x$
dar, din constr., $M_2[x] = 1, \forall x$ $\Rightarrow f_{eq}(M_1, M_2) = 1$

„$\Leftarrow$" $f_{eq}(M_1, M_2) = 1 \Rightarrow M_1[x] = 1, \forall x \Rightarrow M[enc(M)] = 1 \Rightarrow f_{prog}(M) = 1$

+ ce aș mai fi

Scanat cu CamScanner

# Laboratoare AA
## Notații asimptotice

cerca chestii de reducere

↳ cresc la fel de repede

$$\Theta(f(n)) = \{ g: \mathbb{R}^+ \to \mathbb{R}^+ \mid \exists c_1, c_2 > 0, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n) \}$$

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = c, c \neq 0, c \neq \infty \Rightarrow g \in \Theta(f(n))$

↳ cresc maxim la fel de repede

$$O(f(n)) = \{ g: \mathbb{R}^+ \to \mathbb{R}^+ \mid \exists c > 0, \exists n_0 \in \mathbb{N} \text{ astfel încât } \forall n \geq n_0, 0 \leq g(n) \leq c \cdot f(n) \}$$

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = c, c \neq \infty \Rightarrow g \in O(f(n))$

↳ cresc strict mai încet

$$o(f(n)) = \{ g: \mathbb{R}^+ \to \mathbb{R}^+ \mid \forall c > 0, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, g(n) \leq c \cdot f(n) \}$$

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0 \Rightarrow g \in (o(f(n)))$

↳ cresc cel puțin la fel de repede

$$\Omega(f(n)) = \{ g: \mathbb{R}^+ \to \mathbb{R}^+ \mid \exists c > 0, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, c \cdot f(n) \leq g(n) \}$$

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = \begin{cases} c \neq 0 \\ \neq \infty \end{cases} \Rightarrow g \in \Omega(f(n))$

↳ cresc strict mai repede

$$\omega(f(n)) = \{ g: \mathbb{R}^+ \to \mathbb{R}^+ \mid \forall c > 0, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0, c \cdot f(n) \leq g(n) \}$$

$\omega(f(n)) = \Omega(f(n)) \mid \Theta(f($

Dacă $\lim_{n\to\infty} \frac{g(n)}{f(n)} = \infty \Rightarrow g \in \omega(f(n))$

### Syntactic sugars

→ citite de la stânga la dreapta

→ ∀ stg. ⇒ ∃ dreapta

$\Theta(n^2) = O(n^2) + o(n)$

$\forall f \in \Theta(n^2), \exists g \in O(n^2)$ și $\exists h \in o(n)$ a.î. $f(n) = g(n) + h(n)$

---

Ex. 1 → $O(n)$ : $n+2$, $2n+71$, $\log(n)$

→ $\Omega(\log(n))$: $10\log(n)+5$, $n\log(n)$, $n^3$

→ $\Theta(n^2)$: $an^2+bn+d$ $(a>0)$

→ $\omega\left(\frac{1}{n}\right)$: $2$, $n$, $\log(n)$, $n^2$

→ $o(3^n)$ : $n$, $n^2$, $n^p$ $(p \in \mathbb{N})$

Scanat cu CamScanner

## 2 Exemple de constante

→ $\log(n) \in o(\sqrt{n}) \Leftrightarrow \forall c>0, \exists n_0$ a.î. $\forall n \geq n_0, 0 \leq \log(n) \leq c \cdot \sqrt{n}$

$c=1, n_0=1$

→ $100n^2 \in O(2^n) \Rightarrow \exists c>0, \exists n_0$ a.î $\forall n \geq n_0, 100n^2 \leq c \cdot 2^n$

$c=1 \quad n_0=11$

→ $\frac{1}{1000} n^3 \in \Omega(20\sqrt{n}) \Leftrightarrow \exists c>0, \exists n_0$ a.î. $\forall n \geq n_0 \quad c \cdot 20\sqrt{n} \leq \frac{1}{1000} n^3$

$c = \frac{1}{20000}, \quad n_0 = 1$

---

## 3 Exemple de $f$ și $g$

→ $f(n) \underset{\in}{=} O(g^2(n))$

$f(n)=n \qquad g(n)=2n$

→ $f(n) = \omega(\log(g(n)))$

$f(n)=n \qquad g(n)=n$

+ alte exemple de genul

---

## 4 Verificați dacă urm. rel. sunt A/F

→ $\sqrt{n} \in O(\log(n))$ FALS

$\lim_{n\to\infty} \frac{\sqrt{n}}{\log n} \overset{l'H}{=\!=} \lim_{n\to\infty} \frac{\frac{1}{2\sqrt{n}}}{\frac{1}{n}} = \lim_{n\to\infty} 2\sqrt{n} = +\infty \Rightarrow \sqrt{n} \in \omega(\log(n))$

Restul sunt la fel, faci $\lim_{n\to\infty} \frac{g(n)}{f(n)}$, vezi cât dă și care ar fi clasa de complexit rezultată

---

## 5 Reduceți următoarele la o formă mai simplă:

→ $\frac{O(n\sqrt{n})}{\Theta(n)} = \frac{n^{-\infty} \mid n\sqrt{n}}{n} = n^{-\infty} \mid \sqrt{n} = O(\sqrt{n})$

Shaturi:

| | | | |
|---|---|---|---|
| $g \in \Theta(f)$ | $g \simeq f$ | $f(n)$ | lim_inf |
| $g \in O(f)$ | $g \leq f$ | $n^{-\infty} \mid f(n)$ | lim_sup |
| $g \in o(f)$ | $g < f$ | $n^{-\infty} \mid f(n)_-$ | |
| $g \in \Omega(f)$ | $g \geq f$ | $g(n) \mid n^{\infty}$ | |
| $g \in \omega(f)$ | $g > f$ | $f(n)_+ \mid n^{\infty}$ | |

} așa știm cât ar trebui să ne dea

Scanat cu CamScanner

Demonstratie

$f \in O(n\sqrt{n}) \Leftrightarrow \exists c_1 > 0, \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$: $\quad 0 \leq f(n) \leq c_1 n\sqrt{n}$ (1)

$g \in \Theta(n) \hookrightarrow \exists c_2, c_3 > 0, \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_{02}$: $c_2 \cdot n \leq g(n) \leq c_3 \cdot n \Rightarrow \frac{1}{c_3 \cdot n} \leq \frac{1}{g(n)} \leq \frac{1}{c_2 \cdot n}$

Facem raportul $\frac{f(n)}{g(n)}$: $\quad \frac{0 \cdot 1}{c_3 \cdot n} \leq \frac{f(n)}{g(n)} \leq \frac{c_1 n\sqrt{n}}{c_2 \cdot n}$, $\forall n \geq \max(n_{01}, n_{02})$ ($n_0$)

$$0 \leq \frac{f(n)}{g(n)} \leq \frac{c_1}{c_2}\sqrt{n} \Rightarrow \frac{f(n)}{g(n)} \in O(\sqrt{n})$$

($c > 0$)

$\rightarrow \frac{\Theta(n)}{O(\log(n))}$

$f \in \Theta(n) \Rightarrow \exists c_1, c_2 > 0, \exists n_{01} \in \mathbb{N}$ astfel încât $\forall n \geq n_{01}$, $c_1 \cdot n \leq f(n) \leq c_2 \cdot n$ (3)

$g \in O(\log n) \Rightarrow \exists c_3 > 0, \exists n_{02} \in \mathbb{N}$ astfel încât $\forall n \geq n_{02}$, $0 \leq g(n) \leq c_3 \cdot \log(n) \Rightarrow$

$$\Rightarrow \frac{1}{c_3 \cdot \log(n)} \leq \frac{1}{g(n)} < \infty \quad (4)$$

Din (3) și (4) $\Rightarrow$ $0 \leq \frac{c_1 \cdot n}{c_3 \log(n)} \leq \frac{f(n)}{g(n)} \leq \infty$, $\forall n \geq \max(n_{01}, n_{02})$

$\Downarrow$

$$\frac{f(n)}{g(n)} \in \Omega\left(\frac{n}{\log n}\right) \Rightarrow \frac{\Theta(n)}{O(\log(n))} = \Omega\left(\frac{n}{\log(n)}\right)$$

---

6 Verificați

$\rightarrow$ $f(n) = \Omega(\log(n))$, $g(n) \succ O(n)$ $\Rightarrow f(n) = \Omega(\log(g(n)))$

$f(n) \in \Omega(\log(n)) \Rightarrow \exists c_1 > 0, \exists n_{01} \in$ a.î. $\forall n \geq n_{01}$, $c_1 \cdot \log(n) \leq f(n)$ (1)

$g(n) \in O(n) \Rightarrow \exists c_2 > 0 \exists n_{02} \in \mathbb{N}$ a.î. $\forall n \geq n_{02}$ $\quad 0 \leq g(n) \leq c_2 n \mid : c_2$

$$0 \leq \frac{1}{c_2} g(n) \leq n \mid \text{aplicăm log, log} \nearrow$$

$$\log\left(\frac{1}{c_2} g(n)\right) \leq \log n \mid \cdot c_1$$

Scanat cu CamScanner

$c_1 \log\left(\frac{1}{c_2} g(n)\right) \leq c_1 \log(n) \overset{(1)}{\leq} f(n)$ , $\forall n \geq \max(n_{01}, n_{02})$

$c_1 \log\left(\frac{1}{c_2}\right) + c_1 \log(g(n)) \leq f(n)$

$\log\left(\frac{1}{c_2}\right) > 0$

$\Rightarrow f(n) \in \Omega(\log(g(n)))$

$c_1 \log(g(n)) \leq$

+ încă 2 de genul

Scanat cu CamScanner

# Laboratoare AA
## Recurențe

**Obs.** $\Theta(\log_a n) = \Theta(\log_b n)$ , $a>1$, $b>1$

$\log_a n = \frac{\log_b n}{\log_b a} \Rightarrow \Theta(\log_a n) = \Theta\left(\frac{1}{\log_b a} \log_b n\right) = \Theta(\log_b n)$

**Merge Sort:**

$P(n) \rightarrow P(n/2)$, $P(n/2)$ + interclasare

**General:**

$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ → costul adițional de la fiecare apel recursiv

- $a$ – nr. de subprobleme
- $b$ – dimensiunea unei subprobleme

**1 Metoda Master**

Condiții de aplicare:
- $a \geq 1$
- $b > 1$
- $f(n)$ este o funcție asimptotic pozitivă

Cazuri:

I $f(n) = \Theta(n^c)$ cu $c < \log_b a \Rightarrow T(n) \in \Theta(n^{\log_b a})$

II $f(n) = \Theta(n^c \cdot \log^k n)$ cu $c = \log_b a$ și $k \geq 0 \Rightarrow T(n) \in \Theta(n^{\log_b a} \cdot \log^{k+1} n)$

III $f(n) = \Theta(n^c)$ cu $c > \log_b a \Rightarrow T(n) \in \Theta(f(n))$

Ex. $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$

$c = 1$ $a = 2$ $b = 2$

$\log_b a = \log_2 2 = 1$

Cazul II. $\Rightarrow T(n) \in \Theta(n \log n)$

**2 Metoda arborelui**

→ arbore cu noduri $T(n)$

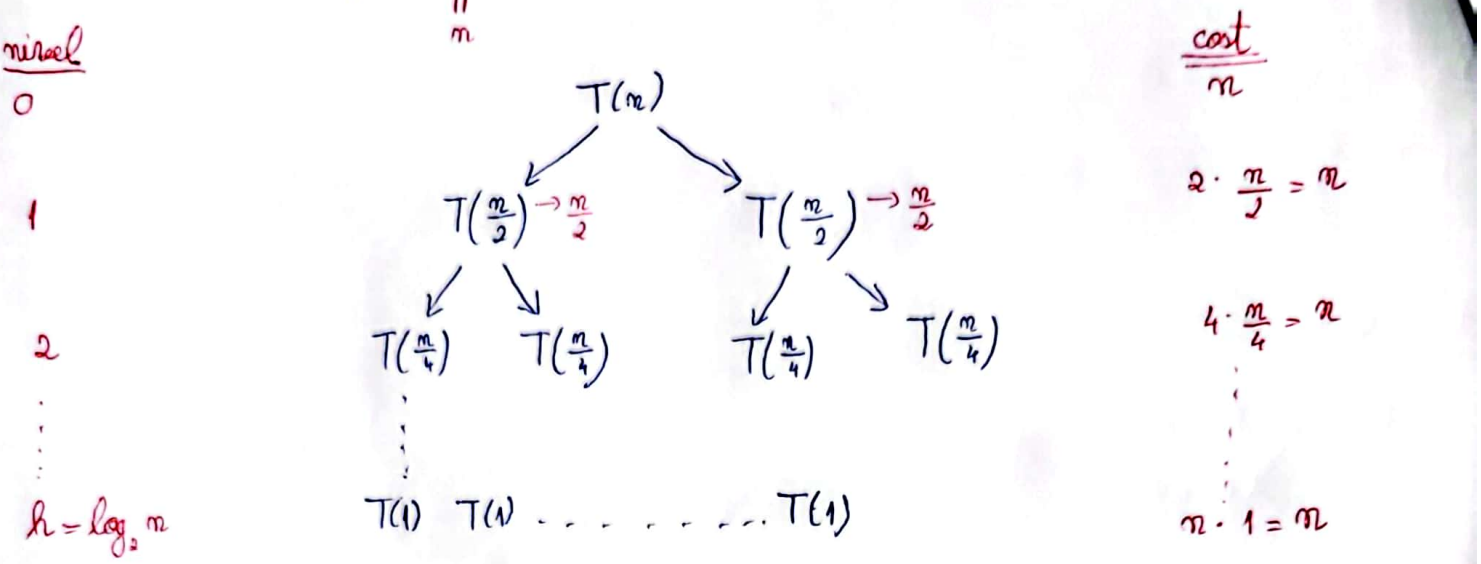
→ $T(n)$ are $a$ copii de forma $T\left(\frac{n}{b}\right)$

→ frunzele sunt $T(1)$ sau $T(0)$

$T(n) = \sum$ costurilor pe fiecare nivel

Scanat cu CamScanner

Ex. $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$
$\Theta(n) = n$




nivel $0 \to \frac{n}{2^0}$

nivel $1 \to \frac{n}{2^1}$

nivel $h \to 1 = \frac{n}{2^h} \Rightarrow h = \log_2 n$

$$T(n) = \sum_{i=0}^{h} \text{cost_nivel_}i = \sum_{i=0}^{h} n = n \cdot (h+1) = n \cdot (\log_2 n + 1) = \Theta(n \log n)$$

## 3 Metoda substituției $T(n) = aT\left(\frac{n}{b}\right) + f(n)$

1) intuim complexitatea
2) demonstrația prin inducție
   i) caz de bază: $T(1)$ sau $T(0)$
   ii) caz general:
   - ipoteza inductivă: presupunem adev. $T\left(\frac{n}{b}\right)$
   - pasul de inducție: dem. adev. pt. $T(n)$

Ex. $\to T(n) = 2T\left(\frac{n}{2}\right) + n$

$T(n) \in \Theta(n \log n) \iff \exists c_1, c_2 > 0, \exists n_0 \in \mathbb{N}$ a.î. $\forall n \geq n_0$ (*)

$$c_1 n \log n \leq T(n) \leq c_2 n \log n$$

Caz de bază: $c_1 \cdot 0 \leq T(1) \leq c_2 \cdot 0$ ok

Caz general:

Scanat cu CamScanner

Presupunem relația (*) adev. pt. $T(\frac{n}{2})$:

$\frac{n}{2}\cdot\log(\frac{n}{2}) \le T(\frac{n}{2}) \le c_2\cdot\frac{n}{2}\log(\frac{n}{2}) \;/\cdot 2$ , ca să am în mijloc o $T(\frac{n}{2})$

(Pi) Dem. rel. (*) adev. pt. $T(n)$

Pornesc de la ce e cu → și vreau să ajung la (*)

$$c_1 n\log(\frac{n}{2}) \le 2T(\frac{n}{2}) \le c_2 n\log(\frac{n}{2}) \;/+n$$

$$\underbrace{c_1 n\log_2 - c_1 n}_{c_1 n\log n} + n \le \underbrace{2T(\frac{n}{2})+n}_{T(n)} \le \underbrace{c_2 n\log n - c_2 n}_{\le c_2 n\log n} + n$$

Vrem să dem. că $\begin{cases} c_1 n\log n \le c_1 n\log n + n\cdot(1-c_1) \\ c_2 n\log n + n\cdot(1-c_2) \le c_2 n\log n \end{cases} \Leftarrow$

$$\Leftarrow \begin{cases} 0 \le n\cdot(1-c_1) \\ n\cdot(1-c_2)\le 0 \end{cases} \Leftarrow \begin{cases} c_1 \le 1 \\ c_2 \ge 1 \end{cases}$$

Pot găsi $c_1=1, c_2=3$ care îndeplinesc cond. ⇒ rel. (*) e adevărată ⇒

$\Rightarrow T(n)\in\Theta(n\log n)$

Exerciții

1 Master

→ $T(n) = 4T(\frac{n}{4})+1$

→ $T(n) = 8T(\frac{n}{2})+2n^2$

→ $T(n) = 4T(\frac{n}{2})+n^2$

2 Binary search prin toate cele 3 metode, ai deja rezolvarea în celelalte laburi

$T(n) = T(\frac{n}{2})+\Theta(1)$

Master

$\log_b a = \log_2 1 = 0$

$g(n) = \Theta(1) = \Theta(n^0) \Rightarrow c=0$ și $k=0$

→ cazul II al teoremei Master ⇒ $T(n)\in\Theta(\log n)$

Scanat cu CamScanner

## Metoda arborelui

| | |
|---|---|
| 0 | $T(n)$ |
| | ↓ |
| 1 | $T(\frac{n}{2})$ |
| | ↓ |
| | $T(\frac{n}{4})$ |
| | ↓ |
| | ⋮ |
| | ↓ |
| $\log_2 n = h$ | $T(1)$ |

$$T(n) = \sum_{i=0}^{h} 1 \cdot c = \sum_{i=0}^{h} c = c \cdot (\log_2 n + 1) \Rightarrow T(n) \in \Theta(\log n)$$

## Metoda Substituției

$-//- \; T(n) \in \Theta(\log n) \; \llcorner \Rightarrow$ $\exists c_1, c_2 > 0$ a.î. $c_1 \log n \leq T(n) \leq c_2 \log n$ (*)

(ii) Presupunem (*) adev. pt. $T(\frac{n}{2})$ : $c_1 \log(\frac{n}{2}) \leq T(\frac{n}{2}) \leq c_2 \log(\frac{n}{2})$

(PI) Dem. (*) pt. $T(n)$

$$\Rightarrow c_1 \log n \leq c_1 \log n - c_1 + c \leq T(\frac{n}{2}) + c \leq c_2 \log n - c_2 + c \leq c_2 \log n$$

Dem $\begin{cases} 0 \leq c - c_1 \\ c - c_2 \leq 0 \end{cases} \llcorner \Rightarrow \begin{cases} c_1 \leq c \\ c_2 \geq c \end{cases} \Rightarrow \exists c_1, c_2$ care indeplinesc condiția $\Rightarrow T(n) \in \Theta(\log n)$

---

## 4 Metoda Substituției

$$T(n) = 2T(\frac{n}{2}) + 1$$

$\log_b a = \log_2 2 = 1$

$f(n) = \Theta(n^0)$

$c < \log_b a$

| cazul I Master $\Rightarrow T(n) \in \Theta(n)$ |
|---|

$-//- \; T(n) \in \Theta(n) \; \llcorner \Rightarrow \exists c_1, c_2 > 0$ a.î. $c_1 n \leq T(n) \leq c_2 n$ (*)

(ii) Presupunem (*) pentru $T(\frac{n}{2})$ : $c_1 \frac{n}{2} \leq T(\frac{n}{2}) \leq c_2 \frac{n}{2} \; / \cdot 2$

Scanat cu CamScanner

$\leq 2T\left(\frac{n}{2}\right) \leq c_2 n \;/+1$

$+1 \leq T(n) \leq c_2 n + 1 \;\wedge$ $\leq c_2 n$ ✗ nu se poate așa

$=$

$-//- \; T(n) \in \Theta(n) \;\lrcorner \Rightarrow T(n) + d \in \Theta(n)$

$\|$

$d \to 0$

$-//- \; T(n) + d \in \Theta(n) \;\lrcorner \Rightarrow \exists c_1, c_2 > 0$ a.î. $\boxed{c_1 n \leq T(n) + d \leq c_2 n}$

(ii) Pres. $c_1 \frac{n}{2} \leq T\left(\frac{n}{2}\right) + d \leq c_2 \frac{n}{2} \;/\cdot 2$

$c_1 n \leq 2T\left(\frac{n}{2}\right) + 2d \leq c_2 n \;/+1-d$

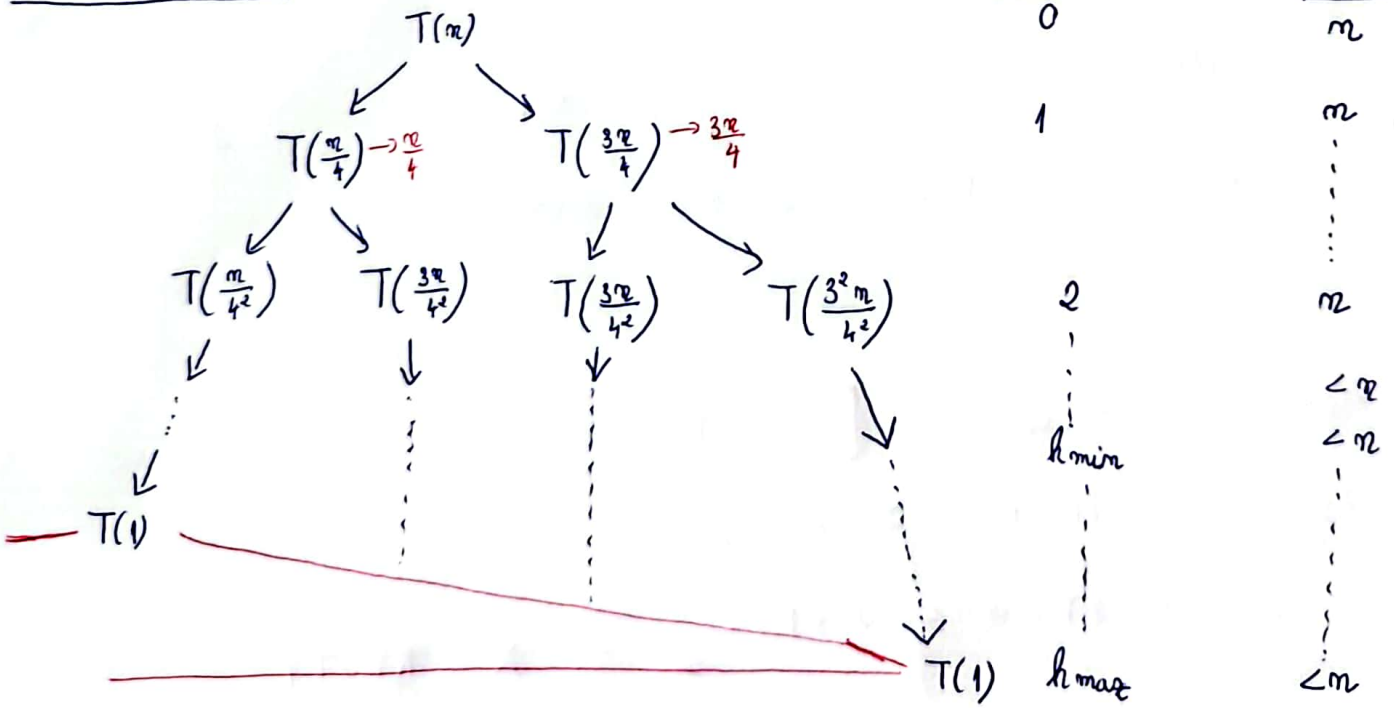
$c_1 n \leq c_1 n + 1 - d \leq T(n) + d \leq c_2 n + 1 - d \leq c_2 n$

Trem $\begin{cases} 0 \leq 1-d \\ 1-d \leq 0 \end{cases} \lrcorner \Rightarrow \begin{cases} d \leq 1 \\ d \geq 1 \end{cases} \to d=1$ îndeplinește condiția $\Rightarrow$ OK inducție $\Rightarrow T(n) \in \Theta(n)$

---

5 $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n$

Metoda arborelui




$\frac{n}{4^{h_{min}}} = 1 \Rightarrow h_{min} = \log_4 n$

$\frac{n}{\left(\frac{4}{3}\right)^{h_{max}}} = 1 \to h_{max} = \log_{\frac{4}{3}} n$

Scanat cu CamScanner

$S_{min} \leq T(n) \leq S_{max}$

- $S_{min}$: costuri de pe fiecare nivel dacă arborele ar avea doar $h_{min}$ nivele
- $T(n)$: suma tuturor costurilor pe nivele
- $S_{max}$: suma costurilor dacă arborele ar avea $h_{max}$ nivele completate complet

$$S_{min} = \sum_{i=0}^{h_{min}} n = n \cdot (1 + h_{min}) = n \log_4 n + n \in \Theta(n \log n)$$

$$S_{max} = \sum_{i=0}^{h_{max}} n = n \cdot (1 + h_{max}) = n \log_{\frac{4}{3}} n + n \in \Theta(n \log n)$$

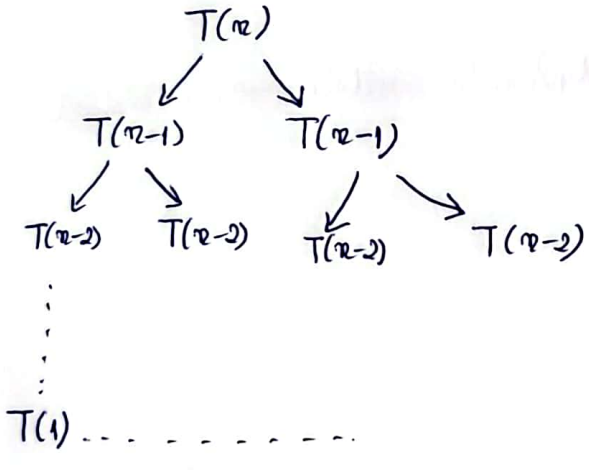
$\Rightarrow T(n) \in \Theta(n \log n)$

---

$T(n) = 2T(n-1) + 1$ prin toate metodele

Master: ✗ (nu are forma bună)

Arbore




| nivel | cost |
|---|---|
| 0 | $2^0$ |
| 1 | $2^1$ |
| 2 | $2^2$ |
| $n-1$ | $2^{n-1}$ |

$$T(n) = \sum_{i=0}^{n-1} 2^i = 2^0 + 2^1 + \dots + 2^{n-1} = 1 \cdot \frac{2^n - 1}{2-1} = 2^n - 1 \Rightarrow T(n) \in \Theta(2^n)$$

Substituție

$T(n) \in \Theta(2^n) \Leftrightarrow \exists c_1, c_2 > 0$ a.î. $\boxed{c_1 \cdot 2^n \leq T(n) \leq c_2 \cdot 2^n}$

(ii): $c_1 2^{n-1} \leq T(n-1) \leq c_2 \cdot 2^{n-1} \;/\cdot 2\;/+1$

$c_1 \cdot 2^n + 1 \leq 2T(n-1) + 1 \leq c^2 \cdot 2^n + 1 \leq c_2 \cdot 2^n$ ✗ KAPUTT

Ca să arătăm că $T(n) \in \Theta(2^n) \Leftrightarrow T(n) + d \in \Theta(2^n) \Leftrightarrow \exists c_1, c_2 > 0$ a.î. $\boxed{c_1 \cdot 2^n \leq T(n) + d \leq c_2 \cdot 2^n}$

(ii): $c_1 \cdot 2^{n-1} \leq T(n-1) + d \leq c_2 \cdot 2^{n-1} \;/\cdot 2$

(PI): $c_1 \cdot 2^n \leq 2T(n-1) + 2d \leq c_2 \cdot 2^n \;/+1-d$

Scanat cu CamScanner

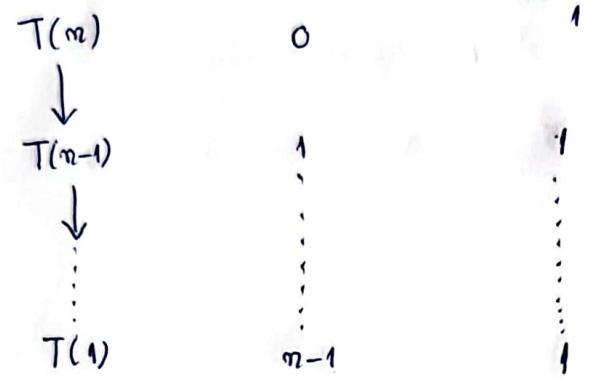
$+1-d \leq T(n)+d \leq c_2 \cdot 2^n + 1 - d \leq c_2 \cdot 2^n$

Vrem $\begin{cases} 0 \leq 1-d \\ 1-d \leq 0 \end{cases} \Rightarrow d=1$ indeplineste conditia $\Rightarrow T(n) \in \Theta(2^n)$

---

$\rightarrow T(n) = T(n-1)+1$

Arbore




$T(n) = \sum_{i=0}^{n-1} 1 = n \Rightarrow T(n) \in \Theta(n)$

---

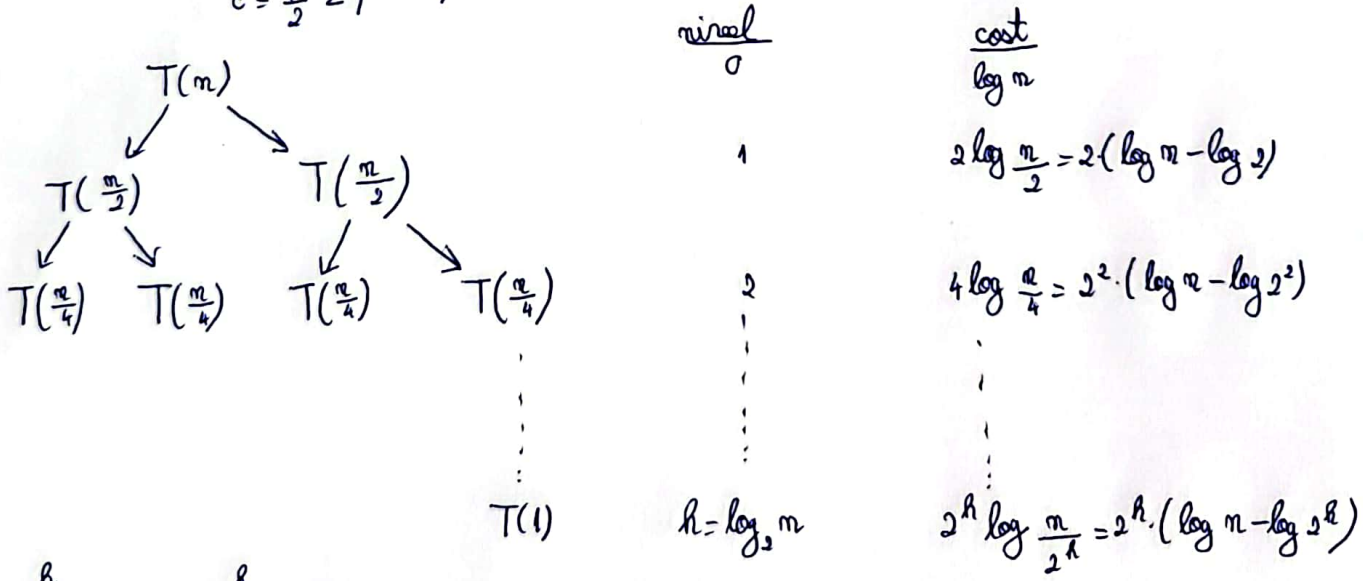
$\rightarrow T(n) = 2T\left(\frac{n}{2}\right) + \log n$ (fine mixte smecherie)

Master: $\log_2 a = 1$

$f(n) = \Theta(\log n) = O(n^{\frac{1}{2}})$ / Master I $\Rightarrow T(n) \in \Theta(n)$

$c = \frac{1}{2} < 1$

Arbore




$T(n) = \underbrace{\sum_{i=0}^{h} 2^i \log n}_{S_1} - \underbrace{\sum_{i=0}^{h} 2^i \log 2^i}_{S_2}$

$S_1 = \log n \sum_{i=0}^{h} 2^i = \log n \cdot (2^{h+1}-1) = \log n \cdot (2^{1+\log_2 n} - 1) = 2n \log n - \log n$

Scanat cu CamScanner

$$S_2 = \sum_{i=0}^{k} i \cdot 2^i$$

$$f(x) = \sum_{i=0}^{k} i \cdot x^i = x \sum_{i=0}^{k} \underbrace{i \cdot x^{i-1}}_{(x^i)'} = x \cdot \sum_{i=0}^{k} (x^i)' = x \left( \sum_{i=0}^{k} x^i \right)' = x \cdot \left( \frac{x^{k+1} - 1}{x-1} \right)' = x \cdot$$

- $\dfrac{(k+1) \cdot x^k \cdot (x-1) - x^{k+1} + 1}{(x-1)^2}$

Pentru $x = 2 \Rightarrow S_2 = f(2) = 2 \cdot (k \cdot 2^k - 2^k + 1) = 2n \log n - 2n + 2$

$T(n) = S_1 - S_2 = \cancel{2n \log n} - \log n \cancel{- 2n \log n} + 2n - 2 = 2n - \log n - 2 \Rightarrow T(n) \in \Theta(n)$

Scanat cu CamScanner

# Laboratoare AA

## TDA-uri

→ descriem comportamentul unei structuri de date
→ asigurăm corectitudinea unei implementări (trebuie să respecte proprietăți)
→ demonstrație prin inducție structurală

TDA: } - constructori = operatori care produc o valoare a tipului de date
→ nulari: A: ␣ → T (ex.: zero, empty)
→ externi: B: S × S → T (ex. constr. pair) } nu e tipul meu de date
→ interni: C: S × T → T (ex. cons)

- Alți operatori: def. prin signatură și prin axiomă
- Proprietăți: relații între operatori ce trb. să fie îndeplinite pt. ca implementarea să fie corectă

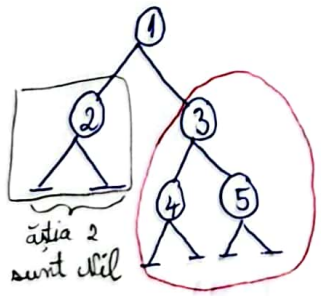
Constructori de bază: suficienți și necesari

## Arbori binari (BTree)

Constructori de bază } Nil: ␣ → BTree
Node: BTree × Integer × BTree → BTree

Node = {t1, e, t2} — t1: arbore stânga; e: etichetă; t2: arbore dreapta

Ex.




{ {Nil, 2, Nil}, 1, {{Nil, 4, Nil}, 3, {Nil, 5, Nil}} }

## Operatori

→ size: BTree → Integer

(S1) size(Nil) = 0
(S2) size({t1, e, t2}) = 1 + size(t1) + size(t2)

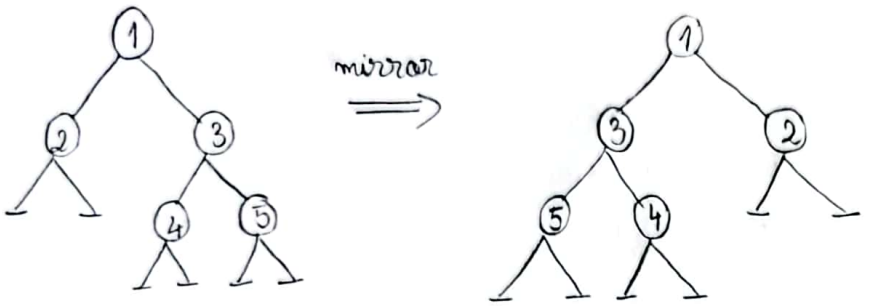
Scanat cu CamScanner

→ height : BTree → Integer

$(H_1)$ height(Nil) = 0

$(H_2)$ height({t₁, e, t₂}) = 1 + max(height(t₁), height(t₂))

→ mirror : BTree → BTree inverseaza subarborele stâng cu subarborele drept pt fiecare nod



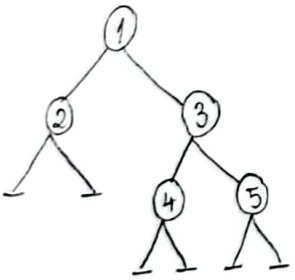

$(M_1)$ mirror(Nil) = Nil

$(M_2)$ mirror({t₁, e, t₂}) = {mirror(t₂), e, mirror(t₁)}

→ flatten : BTree → List intoarce o lista care coresp. unei parcurgeri

EX. NOSTRU: inordine (SRD)

List { empty : ␣ → List (notăm [ ]) → lista in care e adăugat E
cons : E × List → List (notăm (e : l))
concat : List × List → List (l₁ ++ l₂)




flatten ⟹ (2 : (1 : (4 : (3 : (5 : [ ])))))

$(F_1)$ flatten(Nil) = [ ]

$(F_2)$ flatten({t₁, e, t₂}) = flatten(t₁) ++ (e : flatten(t₂))

## Coadă (FIFO)

Ex. → 1, 2, 3 1 e primul element pus

Constr. de bază { void : ␣ → FIFO
enq : FIFO × E → FIFO

Ex. → 1, 2, 3

enq(enq(enq(void, 1), 2), 3)

Scanat cu CamScanner

## Operatori

dequeue : FIFO → FIFO  →1, 2, 3 $\xrightarrow{\text{dequeue}}$ →2, 3  scoate primul element

$$\begin{cases} dequeue(enq(void, e)) = void \\ dequeue(enq(q, e)) = enq(dequeue(q), e) \end{cases}$$

→top : FIFO → E  →1, 2, 3 $\xrightarrow{top}$ 1

$$\begin{cases} top(enq(void, e)) = e \\ top(enq(q, e)) = top(q) \end{cases}$$

→ length : FIFO → Integer

$$\begin{cases} length(void) = 0 \\ length(enq(q, e)) = length(q) + 1 \end{cases}$$

## Map

Constructori de bază
- EmptyMap : ␣ → Map (notăm {})
- put : Map × K × V → Map
  - K: Key
  - V: Value

Operatori

→ get : Map × K → V ∪ {None} → dă valoarea care are ataș o cheie

$$\begin{cases} get(\{\}, k) = None \\ ~~get(put(m, k, v)) =~~ \\ get(put(m, k, v), k) = v \\ get(put(m, k, v), k') = get(m, k') \end{cases}$$

→ update : Map × K × V → Map  dacă există K, actualizează valoare
dacă nu există, adaugă intrarea

$$\begin{cases} update(\{\}, k, v) = put(\{\}, k, v) \\ update(put(m, k, v), k, v') = put(m, k, v') \\ update(put(m, k, v), k', v') = update(m, k', v') \end{cases}$$

Scanat cu CamScanner

~~→ exists(⊥, k) = False Bool~~ → exists : Map × K → Bool

$$\begin{cases} exists(\bot, k) = False \\ exists(put(m, k, v), k) = True \\ exists(put(m, k, v), k') = exists(m, k') \end{cases}$$

→ delete : Map × K → Map

$$\begin{cases} delete(\bot, k) = \bot \\ delete(put(m, k, v), k) = m \\ delete(put(m, k, v), k') = \sout{delete(m, k')} = put(delete(m, k'), k, v) \end{cases}$$

---

List $\begin{cases} [] : _ \to List \\ Cons : E \times List \to List \\ \quad \| \\ \quad (x : l) \end{cases}$

→ length : List → Integer

$$\begin{cases} (L_1)\ length([]) = 0 \\ (L_2)\ length(x : l) = 1 + length(l) \end{cases}$$

→ ++ : List × List → List
(concat)

$$\begin{cases} (C_1)\ [] ++ l = l \\ (C_2)\ (x : l_1) ++ l_2 = (x : (l_1 ++ l_2)) \end{cases}$$

→ reverse : List → List

$$\begin{cases} (R_1)\ reverse([]) = [] \\ (R_2)\ reverse(x : x_0) = reverse(x_0) ++ (x : []) \end{cases}$$

Scanat cu CamScanner

# Laboratoare AA

## Inducție structurală

**Nat** $\begin{cases} zero : _ \to Nat \\ succ : Nat \to Nat \end{cases}$

$\to add : Nat \times Nat \to Nat$

$\begin{cases} add(x, zero) = x \\ add(x, succ(y)) = add(succ(x), y) \end{cases}$

$\to reverse : List \to List$

$\begin{cases} reverse([]) = [] \\ reverse(x : x_s) = reverse(x_s) ++ (x : []) \end{cases}$

---

1) $-//- \forall t \in BTree$, $size(t) = size(mirror(t))$

Dem. prin inducție structurală

Cazul de bază (pt. toți constructorii de bază nulari)

$t \Rightarrow Nil$

$size(Nil) = size(mirror(Nil)) \xleftarrow{M_1}$

$\Leftarrow size(Nil) = size(Nil)$ OK

Cazul general (pt. toți constructorii de bază interni)

$t \Rightarrow \{t_1, e, t_2\}$

Ii (Ipoteza inductivă): presupunem adev. proprietatea pentru $t_1$ și $t_2$:

$size(t_1) = size(mirror(t_1))$

$size(t_2) = size(mirror(t_2))$

Pi (Pas de inducție): dem. adev. proprietatea pt. $\{t_1, e, t_2\}$

$\# size(\{t_1, e, t_2\}) = size(mirror(\{t_1, e, t_2\})) \xleftarrow{M_2, S_2}$

$\Leftarrow 1 + size(t_1) + size(t_2) = size(\{mirror(t_2), e, mirror(t_1)\}) \xleftarrow{S_2}$

$\Leftarrow 1 + size(t_1) + size(t_2) = 1 + size(mirror(t_2)) + size(mirror(t_1)) \xrightarrow{ii}$ OK

---

2) $-//- \forall t \in BTree$, $size(t) = length(flatten(t))$

CB: $t \Rightarrow Nil$

$-//- \; size(Nil) = length(flatten(Nil)) \xleftarrow{F_1}$

$\Leftarrow size(Nil) = length([]) \xleftarrow{S_1, L_1} 0 = 0$ OK

Scanat cu CamScanner

CG : $t \Rightarrow \{t_1, e, t_2\}$

II : $size(t_1) = length(flatten(t_1))$

$size(t_2) = length(flatten(t_2))$

PI : $\neg\neg\ size(\{t_1, e, t_2\}) = length(flatten(\{t_1, e, t_2\})) \xleftarrow{F_2}$

$\xleftarrow{} size(\{t_1, e, t_2\}) = length(flatten(t_1) ++ (e : flatten(t_2))) \xleftarrow{P*}$

P8)

P*) $\forall l_1, l_2 \in List,\ length(l_1 ++ l_2) = length(l_1) + length(l_2)$ → trebuie demonstrată

$\xleftarrow{} size(\{t_1, e, t_2\}) = length(flatten(t_1)) + length(e : flatten(t_2)) \xleftarrow{S_2, L_2}$

$\xleftarrow{} 1 + size(t_1) + size(t_2) = length(flatten(t_1)) + 1 + length(flatten(t_2))$

3) $\forall a, b, c \in Nat,\ add(a, add(b, c)) = add(add(a, b), c)$

Inducție după c

CB : $c \Rightarrow zero$

$\neg\neg\ add(a, add(b, zero)) = add(add(a, b), zero) \xleftarrow{A_1}$

$\xleftarrow{} add(a, b) = add(a, b)$ OK

CG : $c \Rightarrow succ(y)$

II : $add(a, add(b, y)) = add(add(a, b), y)$

PI : $\neg\neg\ add(a, add(b, succ(y))) = add(add(a, b), succ(y)) \xleftarrow{A_2}$

$\xleftarrow{} add(a, add(succ(b), y)) = add(succ(add(a, b)), y) \xleftarrow{P**}$

P**: $add(succ(x), y) = succ(add(x, y))$

$\xleftarrow{} add(a, succ(add(b, y))) = succ(add(add(a, b), y)) \xleftarrow{A_2}$

$\xleftarrow{} add(succ(a), add(b, y)) = succ(add(add(a, b), y)) \xleftarrow{P**}$

$\xleftarrow{} succ(add(a, add(b, y))) = succ(add(add(a, b), y)) \xleftarrow{II}$ adev

Scanat cu CamScanner

P++ : add(succ(x), y) = succ(add(x,y))

Inductie după y

CB : y ⇒ zero

# add(succ(x), zero) = succ(add(x, zero)) ↙ A1 ⇒

↙ ⇒ succ(x) = succ(x) OK

CG : y ⇒ succ(z)

Ii : add(succ(x), z) = succ(add(x,z)) ∀x

Pi : # add(succ(x), succ(z)) = succ(add(x, succ(z))) ↙ A2 ⇒

↙ ⇒ add(succ(succ(x)), z) = succ(add(succ(x), z)) ii ⇒ OK

(succ(x) = x', succ(x) = x')

---

4) ∀l ∈ List, l++[] = l (P4)

CB : l ⇒ []

# []++[] = [] ↙ C1 ⇒ [] = [] Adev.

CG : l ⇒ (x:l)

ii : l++[] = l

Pi : #(x:l)++[] = (x:l) ↙ C2 ⇒

↙ ⇒ (x:(l++[])) = (x:l) ↙ ii ⇒

↙ ⇒ (x:l) = (x:l) Adevărat

---

5) ∀l1, l2, l3 ∈ List l1++(l2++l3) = (l1++l2)++l3

concat(l1, concat(l2, l3)) = concat(concat(l1, l2), l3)

↗ pt că apare primul la toate operațiile de tip concat

Dem. (după l1)

CB l1 ⇒ []

—//— []++(l2++l3) = ([]++l2)++l3 ↙ C1 ⇒

↙ ⇒ l2++l3 = l2++l3 Adevărat

Scanat cu CamScanner

CG $l_1 \Rightarrow (x:l)$

ii: $l ++ (l_2 ++ l_3) = (l ++ l_2) ++ l_3$

Pi: $\xrightarrow{\text{P.a.t.d.}} (x:l) ++ (l_2 ++ l_3) = ((x:l) ++ l_2) ++ l_3 \xleftarrow{C_2}$

$\Leftarrow (x:(l ++ (l_2 ++ l_3))) = (x:(l ++ l_2)) ++ l_3 \xleftarrow{C_2}$

$\Leftarrow (x:(l ++ (l_2 ++ l_3))) = (x:((l ++ l_2) ++ l_3)) \xrightarrow{ii}$ Adev.

6) $\forall l_1, l_2 \in List$, $length(l_1 ++ l_2) = length(l_2 ++ l_1) \xleftarrow{P8}$

$[1,2] ++ [3,4] \Rightarrow [1,2,3,4]$

$[3,4] ++ [1,2] \Rightarrow [3,4,1,2]$

$\Leftarrow length(l_1) + length(l_2) = length(l_2) + length(l_1)$ Adevărat

7) $\forall l_1, l_2 \in List$, $reverse(l_1 ++ l_2) = reverse(l_2) ++ reverse(l_1)$

Inducție după $l_1$

CB $l_1 \Rightarrow [\,]$ (reverse)

$\dashv\!\vdash reverse([\,] ++ l_2) = reverse(l_2) ++ reverse([\,]) \xleftarrow{C_1, R_1}$

$\Leftarrow rev(l_2) = rev(l_2) ++ [\,] \xleftarrow{P_4}$

$\Leftarrow rev(l_2) = rev(l_2)$ Adevărat

CG $l_1 \Rightarrow (x:l)$

ii: $rev(l ++ l_2) = rev(l_2) ++ rev(l)$

Pi: $\dashv\!\vdash rev((x:l) ++ l_2) = rev(l_2) ++ rev(x:l_1) \xleftarrow{C_2, R_2}$

$\Leftarrow rev(x:(l ++ l_2)) = rev(l_2) ++ rev(l_1) ++ (x:[\,]) \xleftarrow{R_2}$

$\Leftarrow rev(l ++ l_2) ++ (x:[\,]) = rev(l_2) ++ rev(l_1) ++ (x:[\,]) \xrightarrow{ii}$ Adev.

Scanat cu CamScanner